

GrantLens

System description — an eligibility-assessment assistant for grant applications.

AI provides the evidence; people make the decision.

Version 2.8 · September 2026
Updated in response to the client's
technical review
AI core: Qwen3-8B running locally (Ollama)

1. Positioning and principles

GrantLens helps grant assessors check each application against the published eligibility criteria **consistently, with verbatim two-way citations, and in a way that can be appealed**. The AI does the time-consuming work (reading, cross-checking, citing, drafting); the officer does the work that matters (assessing, deciding, taking responsibility). All processing — including the language model — runs on the organisation's own server: **applications never leave the premises and there are no API fees**. The demo ships with the 12 verbatim criteria of NSF 22-586 CAREER, 15 labelled sample applications, four real Australian program rule sets and one demonstration fund.

2. The journey of one application

0 Intake — layout and table analysis NEW, requested by client

Accepts .txt / .docx / .pdf or pasted text. PDF/DOCX files are layout-analysed **before retrieval (RAG)**: table regions (budgets, staffing...) are detected and kept row-and-column intact as a single block that can never be split when the text is chunked, so financial figures are not scrambled. Scanned pages without a text layer are detected and flagged "needs OCR" (OCR runs automatically if Tesseract is installed on the server).

0b Denied-party screening (automatic on intake)

Applicant and organisation names are fuzzy-matched (tolerant of misspellings, reordered names and aliases) against the ASIC banned and disqualified persons register (7,212 people, filtered by effective date), the DFAT consolidated sanctions list (3,906 entities, aliases merged), and the ABN business register for existence and status (~20.5 million records, indexed locally). Hits are shown with source, match score and effective status — **they are evidence for the officer to review; the system never rejects an application on its own**.

1 AI assessment — a four-layer pipeline per criterion

- **Layer A · Retrieval (code):** splits the application into passages (keeping character offsets) and retrieves the 3 passages most relevant to the criterion.
- **Layer B · LLM pass 1 — fact extraction:** lists neutral facts only (amounts, percentages, page counts, directorate...) with passage IDs and verbatim key phrases. No judgement.
- **Layer C · LLM pass 2 — judgement:** its only input is the criterion plus the fact list — **it never sees the original text**, so an applicant's grammar or fluency cannot sway the outcome (writing-style bias control). Returns Met / Not met / Unclear / Not addressed plus a confidence level. **NEW** the prompt is enriched with past cases where officers corrected the AI on that same criterion.
- **Layer D · Citation (code):** the quote from the application is cut verbatim by code using the key phrase and string-matched back against the source (100% match expected) — the model is never allowed to write a quote itself.

Results stream live, criterion by criterion, and are all labelled **DRAFT**.

2 Officer allocation — conflict-of-interest screening **NEW, requested by client**

Before an officer takes an application, the server compares the officer against the application using the officer register: **same organisation as the applicant** → **hard block** (no override; the application must be reassigned); same name as the applicant, officer named in the application, or officer with no declared affiliation → warning, and continuing requires a written reason. Everything is written to the audit log.

3 Assessment — no rubber-stamping

The review opens with a **blind spot-check**: the system picks one criterion at random and hides the AI's suggestion until the officer reaches their own conclusion (measuring the officer's independence from the machine). The officer then opens each criterion — seeing the extracted facts, two-way citations (one click highlights the exact sentence in the source), retrieval score and confidence — and **confirms criteria one by one; there is no "approve all" button**. Overriding the AI, or deciding a criterion the AI marked Unclear / Not addressed, requires a written reason. Confirming Met on a criterion that code cannot check requires a verbatim quote from the application, verified mechanically by the server.

4 Sign-off — triggers the feedback loop NEW, requested by client

Sign-off is only possible once every criterion is confirmed; signing faster than 15 seconds per criterion triggers a warning, is logged, and needs a second confirmation. At sign-off, every criterion where **the officer overrode the AI** (or had to decide for it) is automatically added to a **ground-truth store** with the facts, the correct outcome and the reason. The store is (1) used immediately as correction examples for later applications, and (2) exportable in a standard format for periodic fine-tuning — the system aligns more closely with the organisation's standards the more it is used. Applications with a failed or unclear criterion also require a **second-level countersignature by a manager** (not the assessing officer, COI-checked) before any letter is issued.

5 Outcome letter

The AI drafts a plain-English letter (CEFR B1) from the signed-off outcomes: it names each unmet criterion with verbatim quotes from both the guidelines and the application, explains how to supplement the application and how to appeal within 30 days, and follows the organisation's own letter template. The letter is a DRAFT — officers can edit it (the log records "edited"), and it only becomes official once approved, then printed or saved as PDF. Re-assessing a signed-off application requires **Reopen with a reason**.

Throughout — hash-chained audit log: every action (what the AI proposed, who changed what and why, spot-check results, COI warnings, who signed and when) is recorded together with the SHA-256 hash of the previous event. Editing or deleting any entry breaks the chain and is detected with one click — a defensible record for audits and appeals.

3. Interface (single-page web app, 6 areas)

AREA	CONTENTS
Dashboard	Queue and KPIs: rate of officer overrides, verbatim citation match, spot-check agreement with the AI, audit-chain integrity, feedback samples collected.
Applications	Cases by status; the assessment screen shows the criteria matrix, the source application (citations highlighted) and the denied-party screening panel.
Rule sets	Verbatim criteria per grant program with version and approval status, code-guard coverage, and the per-fund measurement status (see section 8).
Bias Lab	Compares two applications with the same facts written in different styles — a fair system must return identical outcomes on every criterion.
Audit log	All events with the hash chain, filterable by application, one-click integrity check.
Data intake	Upload new applications; manual denied-party / ABN look-ups; extract criteria from new grant guidelines (preview, non-verbatim sentences flagged).

4. Technical architecture

COMPONENT	TECHNOLOGY / ROLE
Interface	Self-contained single-page web app (HTML/JS) served by the backend, with live AI progress updates.
Backend	Python FastAPI: a 7-state case workflow; every trust rule (spot-check, mandatory reasons, COI blocks, fast-signing block, letters only after sign-off, verbatim attestation) is enforced on the server — the interface cannot bypass them.
AI core	Qwen3-8B running locally via Ollama (JSON constrained by schema, temperature 0). Switching model or runtime (Qwen 14B, vLLM, any OpenAI-compatible service) is an environment variable, with no change to business logic. A GPU with ≥ 8 GB VRAM gives roughly 1–2 minutes per application.
Document extraction	pdfplumber layout analysis for PDF; DOCX read in true paragraph/table order; fallback OCR for scanned pages.
Screening	Accent-insensitive fuzzy matching; SQLite FTS5 index over 20.5 million ABN records (built once in ~17 minutes, streamed without unpacking the 12 GB archive). Lists can be replaced or extended for the organisation's market (internal, OFAC, EU, UN...).
Storage	SQLite (demo / small MVP) → PostgreSQL for larger deployments; SHA-256 hash-chained audit log; JSONL feedback store.
Evaluation	Automated scoring on 15 labelled applications (accuracy, citation match — 100% expected, pairwise bias score — 100% expected, false passes — 0 expected), plus a labelled test set generated separately for each grant program (section 8); several AI cores can be compared on the same metrics.

5. Mechanisms added after the client review (September 2026)

REQUEST	MECHANISM IMPLEMENTED	TESTED
Financial tables in PDFs break during retrieval	Layout analysis before retrieval; each table kept intact as one block; scanned pages flagged for OCR	5-year budget PDF: table lands in exactly one block, figures intact
Feedback loop to fine-tune the model	Sign-off → "officer overrode AI" samples go to the ground-truth store; injected as few-shot examples immediately and exportable for fine-tuning	Override one criterion and sign: sample stored, next application's prompt contains the correction
Keyword screening before allocation (applicant from the assessor's own organisation)	Officer × application check: same organisation is a hard block; same name / undeclared affiliation warns and requires a reason	An officer from the applicant's university is refused by the server
(Added) Denied-party screening	Fuzzy matching against ASIC + DFAT plus ABN verification, run automatically on intake	A misspelled sanctioned name is still caught (91%); clean names raise no false hits
Verification / anti-cheating: form declares \$500k but the attachment says \$1.5m, fake dates	Cross-document checking: multi-document applications (form + financial report + CV...); code extracts and compares like-for-like figures (budget, revenue, degree year, page count) and implausible dates; mismatches are flagged with context quoted from each source	Test application declaring \$480k with a report stating \$1,520,000: all 3 planted inconsistencies caught; clean application raises no false flags
Standard outputs (rejection / feedback templates)	Outcome letters follow the organisation's own letter template (replace one template file to change the format of every letter)	Template framework in place; awaiting the organisation's real template

6. Rounds 2–3 of the review (September 2026)

ITEM	MECHANISM	VERIFICATION
Multiple grant opportunities + versioning	One versioned rule set per program; each application locks a snapshot of its rule set at first assessment and the log records "LOCKED"; new rule sets are extracted from guidelines in the UI (verbatim extraction → officer review → save as draft → manager approval)	End-to-end: changing the active rule set mid-round does not affect locked cases
Australian program library	Four rule sets quoted verbatim from client-supplied guidelines: Wine Tourism R8, Cyber Security Skills R2, Boosting Female Founders R1, On-farm Water — covering all four program types in the client's classification	Two trap applications built from the client's analysis: cellar-door/wholesale and asset/rebate caps caught correctly
Code guards against false passes	Code re-checks quantitative criteria after the AI; a "Met" that contradicts the figures is overridden to "Not met"; an officer rejecting the warning must give a reason	Real Qwen run: false passes 1/6 → 0/6 , accuracy 24/24 on applications containing every trap
Manager countersignature	Rejected/unclear applications: letters are blocked until a manager (not the assessor, COI-checked) countersigns	End-to-end: self-countersigning blocked (400), non-manager blocked (403)
Supplement rounds + SLA	"Awaiting supplement" state: checklist + deadline, multiple counted rounds; SLA per state, overdue shown in red	End-to-end: supplement round #1 completed and re-assessed automatically

7. Limitations, stated openly

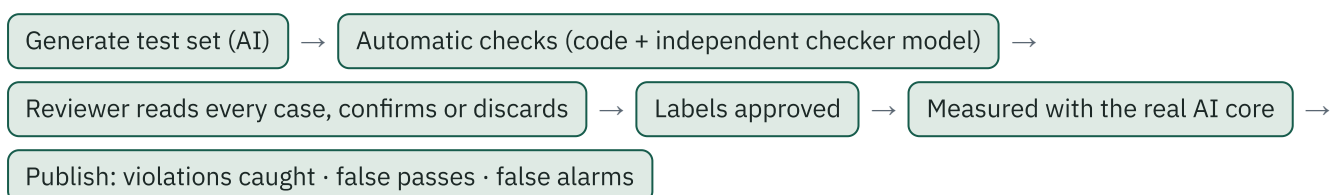
This section exists so that **nobody misreads what the system guarantees**, as requested in the organisation's review.

QUESTION	STRAIGHT ANSWER	MECHANISM IN THE SYSTEM
Do code guards protect every criterion of every program?	No. Only criteria code can express (money / percentage thresholds, prohibited items, exclusion lists). Derived logic, alternative branches, conditional thresholds and conditional exclusions (e.g. "100 or more employees and not compliant with the Workplace Gender Equality Act") cannot be expressed by the compiler.	A 3-level coverage report on every rule set: code-guarded / needs a hand-written guard / LLM + officer only. Each gap is listed with its reason, never silently skipped, and closed with hand-written guards where it matters.
Is a newly loaded program safe straight away?	Only at the basic level. The system compiles constraints from the verbatim criteria, but the "needs a hand-written guard" part still relies on the model and the officer.	Rule-set approval checks minimum guard coverage; below the threshold a manager must explicitly accept the risk with a reason, recorded permanently.
Are there measurements for every program, or just one?	Each program must earn its own figures — no borrowing another program's numbers. The Wine Tourism figure published earlier (3/3 caught, 0/3 false passes) has been withdrawn .	When we extended measurement to the other programs we found that Wine's generated test documents "leaked the answer" (e.g. a test application stating "violating W08"), so that figure is no longer valid. Test-set generation has been rebuilt (section 8) and all six rule sets have been re-measured under the new process — per-program results are in section 8; the Rule sets screen shows each program's measurement status and only publishes figures for programs that have completed the full chain.
Which program does "false passes = 0" apply to?	Only the program measured, using a test set whose labels have been approved. It does not carry over to other programs.	AI-generated test sets start <i>unapproved</i> ; any result on them is marked provisional . A result must also match the exact label set currently approved — otherwise the system shows "re-measurement required".
What about criteria without a code safety net? Can an officer just click through on the AI's word?	No. This is the highest-risk zone, so friction is enforced on the server, not merely displayed.	Confirming Met on such a criterion requires self-verified evidence (the server refuses without it); the criterion is always flagged for attention; the attestation goes into the log under the officer's name; the dashboard counts these for managers.
Can an officer "type enough characters" to pass the attestation gate?	No. The gate does not just count characters.	The attestation must contain a genuine verbatim passage from the application (checked by code, not AI, so there is no judgement error) and cannot be pasted across criteria . For the highest-risk criteria the officer may not borrow the AI's quote

		and must paste their own. Every attestation is in the hash-chained log for later audit.
What about purely qualitative criteria (ABN, GST, licences...)?	Friction applies there too. This is where "the AI says it's fine, the officer clicks along" is most likely.	Met requires a quoted piece of evidence, or explicit confirmation of the exact sentence the AI extracted (only possible after opening it; the attestation type is logged). Criteria already guarded by code are not slowed down — friction is proportional to risk.
The label checker is also AI — what if it is wrong?	It is never trusted on one pass, and it never decides on its own.	Labels are checked by a model from a different family than the system under test (so hard cases are not quietly dropped by the very model being measured, which would inflate the score), using two differently worded questions ("is it satisfied?" / "is it violated?"). The checker only raises flags — a reviewer must confirm or discard every flagged case, and labels cannot be approved while flags remain . Code has veto power when it detects a violation from the figures.
Can generated test cases distort the measurement?	Yes, and it has happened — hence several code-level checks.	(1) The violation is fixed by code first and inserted verbatim into the application, not left to the AI; (2) answer leaks are blocked (criterion codes, self-commentary such as "violates / satisfies"); (3) text that is not an application (the model talking to itself) is rejected; (4) a clean application is used as a control to measure false alarms — without it, a system that rejects everything would score 100%; (5) a reviewer reads every application before labels are approved.

8. Measuring quality program by program

A figure is only published for a program once it has completed the whole chain below. Figures from one program are never used to describe another.



PROGRAM / RULE SET	CODE-GUARDED CRITERIA	MEASUREMENT STATUS	PUBLISHED RESULT
NSF 22-586 CAREER (US)	8 / 12	measured, full chain	caught 1/1 · false passes 0/1 · false alarms 0/1
Wine Tourism R8	5 / 9	measured, full chain	caught 2/2 · false passes 0/2 · false alarms 1/2 (<i>earlier 3/3 figure withdrawn</i>)
Cyber Security Skills R2	4 / 8	measured, full chain	caught 2/2 · false passes 0/2 · false alarms 0/2
Boosting Female Founders R1	3 / 6	measured, full chain	caught 1/2 · false passes 0/2 · referred to officer 1 · false alarms 0/2
On-farm Water	4 / 8	measured, full chain	caught 1/1 · false passes 0/1 · false alarms 0/1
Demonstration fund	1 / 3	measured, full chain	caught 1/1 · false passes 0/1 · false alarms 0/1

What the measurement found — all 6 programs (system under test: qwen3:8b): **8/9** planted violations caught · **0/9 false passes** · 1 case referred to an officer · clean applications decided correctly 8/9 · **1/9 false alarm**.

The guard layer stopped a real AI false pass: in Female Founders F05 the application plainly states "registered income tax exempt entity" (an excluded category), yet the model marked it Met; the code guard caught it, downgraded it to Unclear with a warning and required an officer decision — no false pass got through.

One false alarm, with a known cause: in Wine W06 the clean application states "operates a dedicated physical cellar door located on our property", but the model returned Not met because "there is no information about ownership or lease" — missing information should have been Unclear. Added to the improvement backlog (code-level check: Not met must be backed by a violating fact); the change will be re-measured on all six programs before it is published.

Why the reviewer step is mandatory: in the re-run, 18 AI-generated violation cases passed every automated check, yet the reviewer still discarded **9 of them (5 wrong labels, 4 ambiguous answers)** — generated test sets cannot serve as ground truth unless someone reads every application. **Sample size:** after review, 1–2 violation cases remain per program, so results are **preliminary, per-program evidence, not statistical rates**; the sample is enlarged with real, officer-labelled applications during deployment.

Demo scope: multi-level login and permissions, PostgreSQL, full OCR, and fine-tuning from the feedback store belong to the MVP / Completion phases of the quoted roadmap. Screening currently uses public Australian sources to demonstrate capability — replace them with the organisation's own market lists at deployment.